

CAPSTONE PROJECT EXECUTION PLAN

CSE 600 — Bachelor of Science in Computer Science and Engineering

Department of Computer Science and Engineering

ThreatGPT

LLM-Assisted Cyber Threat Intelligence Summarization and Alert Generation System

University	University of Chittagong
Department	Computer Science and Engineering
Course	CSE 600 — Capstone Project
Semester	6th Semester, B.Sc. CSE
Supervisor	Associate Professor Rezaul Karim
Team Size	3 Students
Project Duration	45 Days
Session	2025–2026

1. Executive Summary

ThreatGPT is a software system designed to assist cybersecurity analysts by automatically summarizing incoming cyber threat intelligence reports and generating structured, actionable security alerts. The core idea is straightforward: organizations today receive massive volumes of threat feeds, vulnerability reports, and security advisories from multiple sources, but they rarely have enough human analysts to read and process all of it in time. ThreatGPT addresses this problem by placing a Large Language Model (LLM) at the center of a pipeline that ingests raw threat data, extracts meaningful insights, classifies the severity of threats, and outputs concise alerts that a security team can act upon immediately.

This document provides a comprehensive, day-by-day project execution plan for three 6th-semester B.Sc. CSE students at the University of Chittagong. The project spans 45 days and is structured into five well-defined phases: environment setup and literature review, data collection and preprocessing, LLM integration and core backend development, frontend dashboard development, and system integration with final testing and documentation. Each phase has clear deliverables, and responsibilities are distributed across the three team members based on complementary skill sets.

2. Project Overview

2.1 Problem Statement

Modern organizations — whether they are banks, hospitals, government agencies, or technology companies — are constantly under the threat of cyberattacks. Security teams rely on Cyber Threat Intelligence (CTI) to stay ahead of attackers. CTI consists of curated information about known threat actors, malware families, attack campaigns, indicators of compromise (IoCs), and vulnerability disclosures. However, the sheer volume of CTI data available from public and commercial sources is overwhelming. A typical security operations center (SOC) may receive hundreds of threat reports per day, written in dense technical language across inconsistent formats.

The result is that analysts spend a disproportionate amount of time reading and manually summarizing these reports instead of responding to actual threats. Important alerts get buried, response times increase, and the organization's overall security posture weakens. ThreatGPT is designed to eliminate this bottleneck by automating the most time-consuming part of CTI analysis: reading, understanding, and summarizing unstructured threat data.

2.2 Proposed Solution

ThreatGPT is a web-based application built around a pre-trained Large Language Model that is prompted in a specialized way to extract and structure threat intelligence from raw text. The system accepts input in the form of pasted threat reports, uploaded text files, or simulated threat feeds. The LLM processes the input and produces a structured output containing a plain-language summary, identified threat type (phishing, ransomware, DDoS, APT, etc.), severity classification (Low, Medium, High, Critical), a list of key indicators of compromise, and a recommended response action.

The output is displayed on a clean, interactive dashboard that allows users to view summaries, filter alerts by severity or threat type, and export reports in PDF or JSON format. The system also maintains a log of all processed alerts so that analysts can review historical data over time.

2.3 Technical Components

- Large Language Model (LLM) via API — Google Gemini or OpenAI GPT for threat summarization and classification
- Python + Flask — backend API server handling all data processing logic
- React.js + Tailwind CSS — responsive frontend dashboard for threat visualization
- SQLite / PostgreSQL — database for storing processed alerts and threat logs
- NLP Preprocessing — text cleaning, tokenization, and keyword extraction before LLM inference
- Alert Engine — rule-based severity scoring combined with LLM output confidence
- PDF Export Module — generating downloadable threat reports for each processed alert

3. Team Structure and Responsibilities

Given that the team consists of exactly three students, roles have been carefully designed so that each member owns a distinct technical layer of the system while also contributing to shared tasks

like documentation, testing, and integration. This prevents any single person from becoming a bottleneck and ensures every team member gains hands-on experience across the full software development lifecycle.

Member	Role	Primary Responsibilities
Student A	Backend & AI Lead	LLM API integration, Flask server, NLP preprocessing pipeline, alert engine logic, database schema
Student B	Frontend & UI Lead	React.js dashboard, threat visualization components, alert filtering, PDF export, responsive design
Student C	Data & DevOps Lead	Dataset collection, text cleaning scripts, system testing, API documentation, deployment, final report writing

All three members are expected to participate equally in weekly supervisor meetings, system integration sessions, and the final poster presentation. The role division above reflects primary ownership, not exclusive responsibility.

4. Step-by-Step 45-Day Execution Plan

The 45-day timeline is divided into five phases. Each phase builds directly on the previous one, and no phase should be skipped or significantly compressed. The plan is intentionally realistic — it accounts for the fact that the team members are full-time students with other academic obligations.

Phase 1: Environment Setup and Literature Review | Days 1–7

Objectives

The first week is the foundation of the entire project. Students should resist the urge to start coding immediately. Instead, this phase is about making sure everyone understands exactly what they are building, why they are building it, and how the tools they will use actually work. A week spent reading and planning at the beginning saves two or three weeks of confusion and rework later.

Day 1–2: Kickoff and Tool Installation

- Hold a project kickoff meeting with the supervisor to align on scope, deliverables, and expectations
- Install Python 3.10+, Node.js 18+, Git, and VS Code on all three machines
- Create a shared GitHub repository with a clear folder structure: /backend, /frontend, /data, /docs
- Set up a shared Notion or Google Docs workspace for taking notes, tracking tasks, and writing documentation collaboratively
- Register for a free-tier API key from Google AI Studio (Gemini) or OpenAI (GPT-3.5/4)

Day 3–5: Literature Review

- Each student reads at least three academic papers or technical articles related to LLM-based threat intelligence, NLP for cybersecurity, and SOC automation
- Recommended starting points: MITRE ATT&CK framework documentation, STIX/TAXII threat intelligence standards, papers on CTI automation using NLP
- Student C compiles a summary of key findings into a shared document that will later feed into the project report
- The team maps out a list of publicly available cyber threat intelligence datasets and sources (CVE feeds, NVD, AlienVault OTX, CISA advisories)

Day 6–7: Architecture Design and Task Assignment

- Draw a full system architecture diagram showing how each component connects — user input, backend API, LLM, database, and frontend dashboard
- Finalize the database schema (tables for alerts, threat_type, severity, timestamps, raw_input, summary, iocs)
- Write a one-page project specification document that everyone agrees on
- Assign GitHub issues to each team member for Phase 2 tasks

Deliverable by Day 7: GitHub repository initialized, architecture diagram complete, literature review notes compiled, API keys obtained and tested with a simple "Hello World" API call.

Phase 2: Data Collection and Preprocessing | Days 8–16

Objectives

This phase focuses on building the data pipeline that feeds content into the LLM. Good threat summarization depends on reasonably clean and well-formatted input text. The team will collect real-world threat intelligence samples, write text preprocessing scripts, and test the pipeline end to end before the LLM is connected.

Day 8–10: Dataset Collection

- Collect at least 50 to 100 real-world cybersecurity threat reports and advisories from public sources
- Sources include: CISA Alerts (cisa.gov), NVD CVE descriptions (nvd.nist.gov), AlienVault OTX pulse reports, Krebs on Security blog posts, and US-CERT advisories
- Student C leads this effort; store all collected documents in the /data/raw folder as plain text files
- Manually annotate 20 of these documents with ground truth labels (threat type, severity, key IoCs) — this will be used to evaluate the LLM output quality later

Day 11–13: Text Preprocessing Pipeline

- Student A writes a Python preprocessing script using the NLTK or spaCy library
- The script should: remove HTML tags and special characters, tokenize text into sentences, extract IP addresses, domain names, CVE IDs, and hash values using regular expressions, and normalize text for LLM consumption
- Write unit tests for the preprocessing functions to make sure edge cases are handled

- Run the pipeline on all 50–100 collected documents and verify the output looks clean and structured

Day 14–16: Prompt Engineering

- This is one of the most critical sub-tasks in the entire project — how you phrase the instruction to the LLM determines the quality of the output
- Student A and Student B collaborate on writing and testing prompt templates
- The prompt should instruct the LLM to return a JSON object with five fields: summary (2–3 sentences), threat_type (one of a predefined list), severity (Low/Medium/High/Critical), ioc_list (array of extracted indicators), and recommended_action (one sentence)
- Test the prompt with at least 10 different threat reports and evaluate whether the LLM output is consistent, accurate, and well-formatted
- Iterate on the prompt until the output quality is satisfactory — expect to go through at least five or six revisions

Deliverable by Day 16: Preprocessing pipeline functional, 50+ threat documents collected and cleaned, prompt template finalized and tested with consistent JSON output.

Phase 3: LLM Integration and Backend Development | Days 17–27

Objectives

Phase 3 is the technical heart of the project. The team will build the Flask API server, integrate the LLM, implement the alert generation engine, and connect the backend to the database. By the end of this phase, the system should be able to accept a raw threat report as input via an API call and return a structured, classified alert as output.

Day 17–19: Flask API Server Setup

- Student A initializes the Flask project with a clean folder structure: /routes, /models, /services, /utils
- Implement the following API endpoints: POST /api/analyze (accepts raw threat text, returns structured alert), GET /api/alerts (returns all stored alerts), GET /api/alerts/<id> (returns a specific alert), DELETE /api/alerts/<id> (removes an alert from the log)
- Set up SQLite as the local database using SQLAlchemy ORM — define the Alert model with all required fields
- Test all endpoints using Postman or curl before adding LLM logic

Day 20–22: LLM Service Integration

- Create a dedicated /services/llm_service.py module that handles all communication with the external LLM API
- The service should: load the finalized prompt template, insert the preprocessed threat text into the prompt, call the LLM API, parse the JSON response, and handle errors gracefully with fallback responses
- Implement retry logic with exponential backoff in case the API call times out or returns an error

- Add response caching using a simple in-memory dictionary so that identical inputs do not trigger redundant API calls

Day 23–25: Alert Engine Development

- The alert engine sits between the LLM output and the database — it validates the LLM response, enriches it with additional metadata, and saves it to the database
- Implement a severity scoring function that cross-checks the LLM's severity label against keyword signals in the raw text (e.g., presence of words like 'zero-day', 'active exploitation', or 'data exfiltration' should bump severity up)
- Add a timestamp, a unique alert ID (UUID), and a processing duration field to every alert before saving
- Write integration tests that send real threat documents through the full pipeline — preprocessing, LLM inference, alert engine, and database storage

Day 26–27: Backend Hardening and API Documentation

- Add input validation to all API endpoints using Flask-Marshmallow or Pydantic
- Implement basic rate limiting on the /api/analyze endpoint to prevent abuse
- Write API documentation using Swagger UI (Flask-RESTX or Flasgger) so the frontend team has a clear reference
- Student C reviews all backend code for quality, tests edge cases, and logs any bugs in GitHub Issues

Deliverable by Day 27: Fully functional backend API with LLM integration, alert engine, database storage, and Swagger documentation. All endpoints tested and passing.

Phase 4: Frontend Dashboard Development | Days 28–37

Objectives

With the backend complete, Student B takes the lead in building the React.js frontend dashboard. The dashboard is the face of ThreatGPT — it is what a security analyst actually sees and interacts with. The goal is a clean, professional, and functional interface that makes the power of the underlying LLM pipeline immediately visible.

Day 28–30: React Project Setup and Layout

- Initialize the React project using Create React App or Vite with Tailwind CSS configured
- Design and implement the main layout: a sidebar for navigation, a top bar with the project title and status indicators, and a main content area
- Create the following pages/views: Home/Dashboard (summary statistics), Analyze (input form), Alerts Log (table view), and Alert Detail (individual alert view)
- Use React Router for navigation between pages

Day 31–33: Core Dashboard Components

- Build the Analyze page: a large textarea where the user can paste or type raw threat text, a submit button that calls the backend `/api/analyze` endpoint, and a loading spinner while the LLM processes the input
- Build the result display component that renders the structured alert output — summary text, threat type badge, severity badge (color-coded: green/yellow/orange/red), IoC list, and recommended action
- Build the Alerts Log page with a sortable and filterable data table using React Table or a lightweight custom implementation — allow filtering by severity, threat type, and date range

Day 34–36: Visualizations and Export

- Add a statistics section to the Home dashboard showing: total alerts processed, breakdown by severity (donut chart), breakdown by threat type (bar chart), and alerts over time (line chart)
- Use Recharts or Chart.js for all visualizations — both are lightweight and easy to integrate with React
- Implement PDF export functionality: when a user clicks 'Export Alert', generate a formatted PDF using jsPDF or react-pdf that includes all alert fields and the university header
- Implement JSON export for developers and analysts who want to pipe alert data into other tools

Day 37: UI Polish and Responsive Design

- Test the entire frontend on different screen sizes and fix any layout issues
- Add loading states, error messages, and empty state illustrations for all pages
- Ensure the color scheme is professional and consistent — use dark blue and white as primary colors, with severity colors reserved for badges only
- Student B presents the completed dashboard to the supervisor for a quick mid-project review

Deliverable by Day 37: Fully functional React.js dashboard connected to the backend API, with threat analysis form, alerts log, visualizations, and PDF/JSON export features working end to end.

Phase 5: Integration, Testing, and Final Documentation | Days 38–45

Objectives

The final phase brings everything together. The team performs full system integration testing, fixes bugs, writes the project report, prepares the poster, and rehearses the viva presentation. This phase requires discipline — resist the temptation to add new features and instead focus entirely on making what already exists work reliably and look professional.

Day 38–40: System Integration and End-to-End Testing

- Merge all frontend and backend code into the main GitHub branch and resolve any merge conflicts
- Perform full end-to-end testing using the 20 manually annotated threat documents from Phase 2 — record whether the LLM's output matches the ground truth labels

- Calculate and document precision and recall for threat type classification and severity assignment
- Fix all critical bugs — anything that causes a crash, produces incorrect output, or breaks the user interface must be resolved before Day 41
- Conduct a user experience walkthrough: have someone who was not involved in building the system try to use it and note any points of confusion

Day 41–42: Performance Optimization and Deployment

- Measure the average response time for the /api/analyze endpoint and optimize if it exceeds 10 seconds
- Deploy the backend to a free cloud platform such as Render or Railway, and the frontend to Vercel or Netlify
- Update the frontend API base URL to point to the deployed backend and verify all features work in the production environment
- Write a README.md file with clear installation, setup, and usage instructions so that the project can be reproduced by anyone

Day 43–44: Project Report and Poster

- Student C leads the writing of the final project report — it should include: abstract, introduction, literature review, system architecture, implementation details, evaluation results, conclusion, and references
- The report should be approximately 15 to 20 pages and written in a formal academic style
- Design the poster using Microsoft PowerPoint or Canva — it should be A1 or A0 size and include the system overview, architecture diagram, key results, and a QR code linking to the GitHub repository
- All three students proofread and review both the report and poster before submission

Day 45: Final Rehearsal and Submission

- Hold a full dry run of the viva presentation — each student should be able to explain their own module in detail and answer questions about any other part of the system
- Prepare a 10-minute live demo of the deployed system for the poster presentation day
- Submit the final report, poster PDF, GitHub link, and deployed application URL to the CPCC committee
- Tag the GitHub repository with v1.0 and write a concise release note summarizing all implemented features

Deliverable by Day 45: Deployed system, final project report, poster, and GitHub repository — all submitted and ready for assessment.

5. Project Timeline at a Glance

Phase	Period	Key Activities	Deliverables
Phase 1: Setup &	Days 1–7	Tool installation, literature review, architecture design	GitHub repo, architecture diagram, literature notes

Phase	Period	Key Activities	Deliverables
Review			
Phase 2: Data & Preprocessing	Days 8–16	Data collection, NLP pipeline, prompt engineering	Cleaned dataset, preprocessing scripts, prompt template
Phase 3: Backend & LLM	Days 17–27	Flask API, LLM integration, alert engine, database	Functional backend API with Swagger docs
Phase 4: Frontend	Days 28–37	React dashboard, charts, PDF export	Live frontend connected to backend
Phase 5: Testing & Docs	Days 38–45	Integration testing, deployment, report, poster	Deployed system, report, poster, GitHub release

6. Evaluation Metrics

The system will be evaluated along two dimensions: technical performance and software quality. Technical performance measures how accurately the LLM identifies threat types and assigns severity levels when compared against the manually annotated ground truth dataset. Software quality measures how well the code is written, how easy the system is to use, and how reliably it runs under normal conditions.

Metric	Target	Measurement Method
Threat Type Classification Accuracy	$\geq 80\%$	Compare LLM output against 20 annotated samples
Severity Assignment Accuracy	$\geq 75\%$	Human review of LLM severity labels vs. ground truth
API Response Time	< 10 seconds	Average measured over 30 test calls
System Uptime	$\geq 99\%$ during demo	Deployed on reliable cloud platform
Report Generation Time	< 3 seconds	Time from button click to PDF download
Code Test Coverage	$\geq 60\%$	Measured using pytest-cov for backend modules

7. Risk Assessment and Mitigation

Risk	Likelihood	Impact	Mitigation Strategy
LLM API rate limits or cost overruns	Medium	High	Use free-tier models (Gemini Flash), implement caching, test with small batches
LLM output is inconsistent or poorly formatted	Medium	High	Enforce strict JSON output format in prompt, add output validation layer

Risk	Likelihood	Impact	Mitigation Strategy
One team member falls behind	Low	Medium	Weekly check-ins with supervisor, shared task tracking on GitHub Issues
Deployment fails close to deadline	Low	High	Test deployment from Day 40 onwards, keep a local demo as backup
Insufficient publicly available threat data	Low	Medium	Use synthetic examples and CISA/NVD feeds as fallback sources

8. Conclusion

ThreatGPT is a well-scoped, technically meaningful, and practically relevant capstone project that gives three students a genuine opportunity to work with Large Language Models, build a full-stack web application, and solve a real problem in the cybersecurity domain — all within a tight but manageable 45-day window.

The step-by-step plan outlined in this document is designed to be both ambitious and achievable. Each phase has been sized to match the working capacity of part-time student developers, and each deliverable is concrete enough that progress can be assessed objectively at every supervisor meeting. If the team follows this plan closely and communicates proactively when they run into problems, there is every reason to expect a polished, functional, and impressive final product.

The skills the students will develop through this project — LLM API integration, NLP preprocessing, REST API design, React development, cloud deployment, and technical writing — are among the most in-demand skills in the software industry today. ThreatGPT is not just a university assignment; it is a portfolio project that every team member can be proud to show to future employers.

Prepared by: Associate Professor Rezaul Karim

Department of Computer Science and Engineering, University of Chittagong
rezaul.cse@cu.ac.bd | CSE 600 Capstone Project, Session 2025–2026